

Politecnico di Torino  
*Progetto di Programmazione in RUST — A.A. 2025-2026*

# GEORUGGINE

Sistema client/server per la geolocalizzazione e la comunicazione con una flotta di veicoli

## Manuale del Progettista

*Descrizione tecnica dell'architettura, delle scelte implementative e della valutazione del progetto*

Gruppo: G11  
Componenti: Melillo Maurizio – s360164, Alex martinelli - S357881, Michele Savino - s361490,  
Andrea Sangineto - s362052

## Indice

|                                                               |    |
|---------------------------------------------------------------|----|
| Indice .....                                                  | 2  |
| 1. Introduzione .....                                         | 4  |
| 1.1 Scopo del documento .....                                 | 4  |
| 1.2 Obiettivi del progetto .....                              | 4  |
| 2. Architettura generale del sistema .....                    | 4  |
| 2.1 Struttura del workspace Cargo .....                       | 4  |
| 2.2 Modello client/server e protocollo di comunicazione ..... | 5  |
| Messaggi Client ->Server (enum ClientMessage) .....           | 5  |
| Messaggi Server ->Client (enum ServerMessage) .....           | 5  |
| 2.3 Pattern architetturale comune: GUI + thread di rete ..... | 5  |
| 3. Il crate condiviso "shared" .....                          | 6  |
| 3.1 Costanti di specifica .....                               | 6  |
| 3.2 Sicurezza delle credenziali .....                         | 6  |
| 3.3 Normalizzazione e validazione dello username .....        | 7  |
| 3.4 Funzioni geografiche .....                                | 7  |
| 3.5 Test automatici del crate condiviso .....                 | 7  |
| 4. Persistenza: il database SQLite .....                      | 8  |
| 4.1 Schema .....                                              | 8  |
| 4.2 Configurazione della connessione .....                    | 8  |
| 5. Il server .....                                            | 8  |
| 5.1 Architettura interna e concorrenza .....                  | 9  |
| 5.2 Ciclo di vita di una connessione .....                    | 9  |
| 5.3 Macchina a stati del veicolo .....                        | 9  |
| 5.4 Analisi del movimento .....                               | 10 |
| 5.5 Comunicazione e comandi amministrativi .....              | 10 |
| 5.6 Interfaccia grafica dell'operatore .....                  | 11 |
| 5.7 Log delle prestazioni (CPU/RAM) .....                     | 11 |
| 6. Il client .....                                            | 11 |
| 6.1 Architettura interna e concorrenza .....                  | 11 |
| 6.2 Autenticazione .....                                      | 12 |
| 6.3 Emulazione del movimento .....                            | 12 |
| 6.4 Interfaccia grafica dell'autista .....                    | 12 |
| 6.5 Visualizzazione cartografica .....                        | 13 |
| 7. Scelte di progettazione per le prestazioni .....           | 13 |
| 7.1 Consumo di CPU .....                                      | 13 |
| 7.2 Dimensione dell'eseguibile .....                          | 14 |

|                                                  |    |
|--------------------------------------------------|----|
| 8. Test automatici e dataset di prova .....      | 14 |
| 9. Multiplatforma .....                          | 15 |
| 10. Limiti noti e possibili sviluppi futuri..... | 15 |
| 11. Conclusioni .....                            | 16 |

# 1. Introduzione

## 1.1 Scopo del documento

Questo manuale del progettista descrive come il sistema è stato realizzato, quali scelte architetturali sono state adottate e perché, come sono organizzati i moduli di codice e come le singole funzionalità richieste dalla traccia sono state implementate concretamente.

## 1.2 Obiettivi del progetto

La traccia richiede la realizzazione di un'applicazione client/server in Rust per la gestione della geolocalizzazione e della comunicazione con una flotta di veicoli. In sintesi il sistema soddisfa le seguenti specifiche:

- permette a più utenti di registrarsi e autenticarsi con account e password;
- riceve ogni 30 secondi la posizione geografica (emulata) di ciascun utente connesso;
- gestisce lo stato di ogni utente (sconnesso / fermo / in movimento), con transizioni automatiche basate sulle coordinate ricevute;
- offre una strategia di emulazione del movimento dei veicoli;
- calcola, su richiesta, l'analisi del movimento di un utente (tragitto, velocità media, tempi di moto e sosta) su intervalli programmabili (giorno / settimana / mese);
- supporta la comunicazione testuale server -> utenti (broadcast o diretta) e utente -> server;
- gira su almeno due piattaforme distinte tra quelle indicate nel documento di specifica;
- presta attenzione al consumo di CPU e alla dimensione dell'eseguibile, con un log periodico (ogni due minuti) del tempo di CPU utilizzato dal server.

Le sezioni seguenti mostrano come ciascuno di questi punti è stato tradotto in codice.

# 2. Architettura generale del sistema

## 2.1 Struttura del workspace Cargo

Il progetto è organizzato come un workspace Cargo con tre crate:

| Crate  | Ruolo                                                                                                                                                      |
|--------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
| shared | Libreria condivisa: tipi del protocollo, costanti di specifica, funzioni di validazione, hashing e calcolo geografico usate sia dal client sia dal server. |
| server | Binario eseguibile della console operatore: rete, database, logica di dominio, GUI grafica.                                                                |
| client | Binario eseguibile del terminale di bordo: rete, emulazione del movimento, GUI grafica.                                                                    |

Condividere i tipi del protocollo e le regole di validazione in un unico crate (shared) evita che client e server possano “divergere” nell'interpretazione dei messaggi o nei controlli sui dati (ad esempio le regole di validità dello username nella fase di login, sono scritte una sola volta e richiamate da entrambi).

## 2.2 Modello client/server e protocollo di comunicazione

La comunicazione avviene su TCP con un protocollo applicativo testuale: ogni messaggio è un oggetto JSON serializzato con `serde_json` e terminato da un carattere di <a capo> ("line-delimited JSON"). Questa scelta rende il framing sopra TCP banale (basta leggere riga per riga con un `BufReader`) e i messaggi facilmente ispezionabili durante il debug.

### Messaggi Client ->Server (enum ClientMessage)

| Variante                         | Significato                                                                                                                                                                                                                    |
|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Register { user, password_hash } | Richiesta di creazione di un nuovo account.                                                                                                                                                                                    |
| Login { user, password_hash }    | Richiesta di autenticazione con un account esistente.                                                                                                                                                                          |
| PositionUpdate { lat, lon }      | Posizione emulata, inviata ogni 30 secondi (INTERVALLO_INVIO_SEC).                                                                                                                                                             |
| RichiestaAnalisi(Intervallo)     | Richiede l'analisi del proprio movimento su Giorno / Settimana / Mese.                                                                                                                                                         |
| ChatMessage { testo }            | Messaggio di testo indirizzato al server/operatore.                                                                                                                                                                            |
| PercorsoAvviato                  | Segnala l'inizio (o il riavvio) dell'emulazione: il server azzera il punto di partenza della traccia mostrata sulla mappa, così un riavvio del percorso non disegna una riga di ritorno verso il punto di partenza precedente. |
| Logout                           | Disconnessione esplicita: la sessione applicativa si chiude, ma il socket TCP resta aperto.                                                                                                                                    |

### Messaggi Server ->Client (enum ServerMessage)

| Variante                                                                                          | Significato                                                              |
|---------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| AuthResult { success, messaggio }                                                                 | Esito del login.                                                         |
| RegisterResult { success, messaggio }                                                             | Esito della registrazione.                                               |
| AnalisiResult { intervallo, tragitto_km, velocita_media_kmh, movimento_sec, pause_sec, campioni } | Risultato del calcolo di analisi.                                        |
| StatoAggiornato { stato, lat, lon }                                                               | Notifica in tempo reale dello stato logico e dell'ultima posizione nota. |
| MessaggioDalServer { testo, broadcast }                                                           | Messaggio di testo dal server, distinto in broadcast o diretto.          |
| LogoutOk                                                                                          | Conferma che il server ha chiuso la sessione applicativa dopo un Logout. |

L'uso di enum Rust con `#[derive(Serialize, Deserialize)]` per rappresentare l'intero protocollo consente al compilatore di verificare staticamente, in entrambi i binari, che ogni ramo dei messaggi sia gestito (i match sulle varianti sono esaustivi): un nuovo tipo di messaggio dimenticato in un ramo del codice risulta in un errore di compilazione, non in un bug scoperto a runtime.

## 2.3 Pattern architetturale comune: GUI + thread di rete

Sia il client sia il server condividono lo stesso schema architetturale, imposto da un vincolo tecnico preciso: la libreria grafica scelta (egui/eframe, immediate-mode GUI, multiplatforma) richiede di girare sul thread principale del processo su diverse piattaforme. Tutta la parte di rete, invece, è scritta con Tokio (runtime asincrono) e deve poter attendere su I/O senza bloccare il disegno dell'interfaccia.

La soluzione adottata in entrambi i binari è:

- un thread dedicato, con un proprio runtime Tokio, che si occupa esclusivamente di rete (e, lato server, di database e logica di dominio);
- il thread principale che esegue esclusivamente `eframe::run_native`, cioè il ciclo di disegno della GUI;
- due canali per far comunicare i due mondi senza condividere memoria mutabile: un canale `tokio::sync::mpsc` per i comandi GUI ->rete, e un canale `std::sync::mpsc` per gli eventi rete ->GUI.

Questa separazione evita due problemi tipici: bloccare il disegno dell'interfaccia in attesa di I/O di rete (che la renderebbe percepibile come “bloccata”), e dover portare tipi non Send/Sync della GUI dentro il runtime asincrono. La differenza tra i due binari è nel tipo di runtime Tokio: il server usa un runtime multi-thread (`new_multi_thread`), perché deve gestire più connessioni contemporaneamente eseguendo un task asincrono per ciascuna; il client usa un runtime a thread singolo (`new_current_thread`), perché gestisce una sola connessione e un singolo task di simulazione del percorso, per cui un secondo runtime multi-thread sprecherebbe solo memoria e thread di sistema operativo.

Sul lato “eventi verso la GUI” entrambe le applicazioni notificano un repaint esplicito del contesto egui (`ctx.request_repaint()`) quando arriva un evento di rete, invece di ridisegnare in continuazione a frequenza fissa: è la prima delle scelte pensate per contenere il consumo di CPU richiesto dalla specifica (approfondito nel capitolo 7).

### 3. Il crate condiviso “shared”

Il crate `shared` (`lib.rs`) raccoglie tutto ciò che client e server devono concordare bit per bit: i tipi del protocollo (visti nel capitolo 2), le costanti dettate dalla specifica, e alcune funzioni pure senza effetti collaterali, facilmente testabili in isolamento.

#### 3.1 Costanti di specifica

| Costante                          | Valore | Significato                                                                              |
|-----------------------------------|--------|------------------------------------------------------------------------------------------|
| <code>INTERVALLO_INVIO_SEC</code> | 30     | Ogni quanti secondi il client trasmette la propria posizione.                            |
| <code>TIMEOUT_FERMO_SEC</code>    | 180    | Dopo quanti secondi di coordinate invariate un utente passa da “in movimento” a “fermo”. |
| <code>PORTA_DEFAULT</code>        | 8080   | Porta TCP di default del server.                                                         |

Definire questi valori una sola volta in `shared`, invece di duplicarli come letterali in client e server, garantisce che entrambi i lati del sistema concordino esattamente sulla stessa semantica della macchina a stati.

#### 3.2 Sicurezza delle credenziali

La password non transita mai in chiaro sulla rete né viene mai scritta in chiaro nel database. La funzione `hash_password` calcola lato client un digest SHA-256 (crate `sha2`) usando lo username normalizzato:

```
SHA256( "georuggine:v1:" || username_normalizzato || ":" || password )
```

Questo assicura che due utenti con la stessa password non producano lo stesso hash, mitigando attacchi con tabelle precalcolate (rainbow table) condivise tra account diversi. Se una persona non autorizzata copia il tuo file `georuggine.db` e cerca le password usando le proprie tabelle precalcolate, l'operazione fallisce. L'aggiunta dello username genera un hash assente in quelle liste predefinite. Va detto per completezza tecnica che, trattandosi di un digest crittografico senza costo computazionale regolabile (a differenza di algoritmi pensati apposta per le password come `bcrypt`/`argon2`), questa soluzione è adeguata al contesto didattico del progetto ma non rappresenterebbe una scelta sufficiente per un sistema di produzione esposto pubblicamente.

### 3.3 Normalizzazione e validazione dello username

`Normalizza_username` applica `trim` e `lower-case`, così che gli username siano trattati come case-insensitive e senza spazi ai bordi sia in fase di confronto sia di memorizzazione. `Valida_username` applica le regole della specifica riportate anche nel manuale utente (3-24 caratteri, solo lettere, cifre, `'_'`, `':'`, `'-'`) e viene chiamata sia dal client, per dare un feedback immediato senza un giro di rete, sia dal server, come controllo autorevole che non può essere aggirato da un client malformato o modificato.

### 3.4 Funzioni geografiche

`Haversine_km` implementa la formula dell'emisfero per calcolare la distanza in chilometri tra due coppie di coordinate WGS84, usata dal server per accumulare il tragitto percorso durante l'analisi del movimento. `Coordinate_valide` verifica che latitudine e longitudine siano numeri finiti e compresi nei range validi (`[-90,90]` e `[-180,180]`), scartando pacchetti corrotti o inseriti manualmente in modo errato prima che vengano scritti nel database. `Formatta_durata` converte un numero di secondi in una stringa leggibile ("`1h 05m 30s`"), riusata identicamente da entrambe le interfacce grafiche per mostrare gli stessi tempi con lo stesso formato.

### 3.5 Test automatici del crate condiviso

Il modulo `#[cfg(test)]` di `lib.rs` copre i tre punti più delicati perché condivisi da entrambe le applicazioni:

- Verifica che l'hash sia insensibile a maiuscole/spazi nello username (Mario e mario producono lo stesso digest), che utenti diversi con la stessa password producano digest diversi, e che la lunghezza sia sempre 64 caratteri esadecimali;
- Verifica che `haversine_km` restituisca una distanza plausibile (~44 km, con tolleranza) tra le coordinate di Torino e Asti, usate anche nel tracciato di prova `percorso.csv`;
- Verifica che username con spazi ai bordi vengano accettati e normalizzati, mentre username troppo corti o con spazi interni vengano rifiutati.

## 4. Persistenza: il database SQLite

Il server usa SQLite (crate rusqlite) come unico archivio persistente, nel file georuggine.db creato automaticamente al primo avvio nella directory di lavoro.

### 4.1 Schema

```
CREATE TABLE Utenti ( id INTEGER PRIMARY KEY, username TEXT NOT NULL UNIQUE, password_hash TEXT NOT NULL, creato_il DATETIME DEFAULT CURRENT_TIMESTAMP ); CREATE TABLE Posizioni ( id INTEGER PRIMARY KEY, id_utente INTEGER NOT NULL, lat REAL NOT NULL, lon REAL NOT NULL, timestamp DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY(id_utente) REFERENCES Utenti(id) ); CREATE INDEX idx_posizioni_utente_ts ON Posizioni(id_utente, timestamp);
```

Ogni posizione ricevuta viene inserita come nuova riga in Posizioni, senza mai sovrascrivere lo storico: questo è ciò che rende possibile, in un secondo momento, ricalcolare l'analisi del movimento su un intervallo qualunque semplicemente filtrando per timestamp, senza dover mantenere contatori incrementali che andrebbero gestiti con attenzione ad ogni transizione di stato. L'indice composto su (id\_utente, timestamp) rende immediata sia l'estrazione ordinata della traccia di un utente sia il filtro per intervallo usato dall'analisi, evitando una scansione completa della tabella ad ogni richiesta.

### 4.2 Configurazione della connessione

Ogni connessione SQLite aperta dal server (apri\_db) viene configurata con:

- PRAGMA journal\_mode = WAL: consente letture concorrenti mentre una scrittura è in corso, necessario perché più task asincroni (una connessione TCP per utente) e la GUI dell'operatore accedono al database in parallelo;
- synchronous = NORMAL: un compromesso tra sicurezza dei dati e velocità di scrittura adeguato al WAL journaling, per non pagare in prestazioni il costo di una sincronizzazione sul disco ad ogni singola posizione ricevuta (una ogni 30 secondi per veicolo, ma su più veicoli in parallelo);
- foreign\_keys = ON: SQLite non applica i vincoli di chiave esterna di default; va abilitato esplicitamente perché il legame Posizioni.id\_utente -> Utenti.id sia effettivamente controllato;
- busy\_timeout(5s): se il database è momentaneamente occupato da un'altra connessione, l'operazione attende fino a 5 secondi invece di fallire subito con "database is locked";

La GUI del server apre una propria connessione di sola lettura (conn\_lettura), separata da quella usata dal thread di rete per i comandi amministrativi: grazie al journaling WAL questo è sicuro e permette alla GUI di interrogare direttamente il database (elenco utenti registrati, ricostruzione della traccia sulla mappa) senza dover passare ogni singola lettura attraverso il canale verso il thread di rete.

## 5. Il server

Il server implementa la console dell'operatore: accetta connessioni TCP in ingresso, gestisce autenticazione e stato di ciascun veicolo, calcola le analisi e mostra tutto attraverso un'interfaccia grafica egui/eframe.

## 5.1 Architettura interna e concorrenza

Il cuore del server è il Registro delle sessioni attive, una struttura condivisa `Arc<Mutex<HashMap<String, Sessione>>>` indicizzata per username. Ogni Sessione contiene il canale di invio (tx) verso il task che scrive sul socket di quell'utente, lo stato corrente, l'ultima posizione nota, l'istante di connessione e l'istante di inizio dell'attuale tragitto (usato per non ricollegare sulla mappa la traccia di un tragitto appena riavviato a quella del tragitto precedente).

Per ogni connessione accettata dal listener TCP viene avviato (`tokio::spawn`) un task asincrono indipendente (`gestisci_connessione`), così le connessioni non si bloccano a vicenda: un client lento nello scrivere non rallenta gli altri. All'interno di ogni connessione, la scrittura verso il socket è delegata a un secondo task dedicato che consuma un canale `mpsc`, così che un client lento in lettura (scrittura bloccata sul socket) non impedisca al task principale di continuare a leggere e processare i messaggi in arrivo dallo stesso client.

Ogni task di connessione usa un `tokio::select!` per attendere contemporaneamente due eventi: una nuova riga dal socket, oppure lo scadere di un timer periodico (ogni 5 secondi) usato per rivalutare la transizione “in movimento ->fermo” anche quando, per qualche motivo, non arrivano nuovi pacchetti dal client.

## 5.2 Ciclo di vita di una connessione

Schematicamente, una sessione utente attraversa questi passi:

- il server crea la connessione.
- il client apre il socket TCP; il server accetta la connessione e avvia il task dedicato;
- il client invia Register oppure Login; il server valida username/password rispetto al database e, in caso di successo del login, inserisce una nuova Sessione nel Registro con stato iniziale Fermo;
- il server rifiuta un secondo login con lo stesso username già online (“Utente già connesso da un altro dispositivo”), così da avere sempre una sola sessione attiva per account;
- ad ogni PositionUpdate il server scrive la posizione nella tabella “Posizioni”, aggiorna la macchina a stati (vedi 5.3) e rimanda al client un StatoAggiornato con stato e coordinate;
- ad ogni RichiestaAnalisi il server calcola i dati (vedi 5.4) e li restituisce con AnalisiResult;
- un Logout esplicito rimuove la sessione dal Registro e riporta lo stato a Sconnesso, ma lascia il socket TCP aperto, così il client può tornare alla schermata di login senza dover riconnettersi da zero;
- alla chiusura del socket, il server rimuove comunque la sessione dal Registro, così l'utente torna a comparire come offline nella GUI dell'operatore.

## 5.3 Macchina a stati del veicolo

Lo stato (Sconnesso / Fermo / InMovimento) è calcolato e validato interamente lato server, come richiesto dalla specifica, confrontando ogni nuova coppia di coordinate con l'ultima ricevuta.

| Transizione          | Condizione                                                                       |
|----------------------|----------------------------------------------------------------------------------|
| Fermo ->In movimento | la coordinata ricevuta differisce dalla precedente (primo cambiamento rilevato). |

| Transizione          | Condizione                                                                                        |
|----------------------|---------------------------------------------------------------------------------------------------|
| In movimento ->Fermo | sono trascorsi almeno 180 secondi (TIMEOUT_FERMO_SEC) dall'ultimo cambio di coordinata effettivo. |

La transizione verso “fermo” è verificata in due punti indipendenti: subito dopo aver ricevuto un PositionUpdate (nel caso il client continui a inviare la stessa posizione, come durante una sosta emulata), e periodicamente ogni 5 secondi dal timer del tokio::select!, così da rilevare la transizione anche se, per un istante, non arriva alcun pacchetto. In entrambi i casi lo stato aggiornato viene sia scritto nel Registro condiviso (visibile subito alla GUI) sia notificato al client con un nuovo StatoAggiornato.

## 5.4 Analisi del movimento

La funzione analizza esegue un'unica interrogazione SQL che estrae, per l'utente e l'intervallo richiesti, tutte le posizioni ordinate cronologicamente, poi le scandisce una sola volta linearmente:

```
SELECT lat, lon, strftime('%s', timestamp) FROM Posizioni WHERE id_utente = ?1 AND
timestamp >= <inizio_intervallo> ORDER BY timestamp ASC
```

L'inizio dell'intervallo è espresso direttamente in SQL (date('now','start of day') per il giorno corrente, date('now','weekday 0','-6 days') per il lunedì della settimana ISO corrente, date('now','start of month') per il mese corrente), delegando al motore SQLite il calcolo di calendario invece di reimplementarlo in Rust.

Per ogni coppia di campioni consecutivi il calcolo distingue tre casi:

- se la distanza dal campione precedente supera 10 metri (SOGLIA\_MOVIMENTO\_KM), l'intervallo di tempo trascorso viene contato come movimento e la distanza (haversine\_km) sommata al tragitto totale;
- se la distanza è inferiore a 10 metri, l'intervallo viene contato come sosta: questa soglia assorbe il “rumore” di un GPS emulato (o reale) che oscilla leggermente pur restando fermo, evitando di conteggiare come movimento micro-variazioni non significative;
- se il tempo trascorso tra due campioni supera 300 secondi (GAP\_MAX\_SEC), l'intervallo non viene conteggiato né come moto né come sosta: un buco così ampio indica che l'utente era disconnesso, e sommarlo a uno dei due totali falserebbe il risultato.

La velocità media è infine ricavata come tragitto\_km / (movimento\_sec / 3600), cioè rapportata al solo tempo effettivo di moto e non all'intera durata dell'intervallo, in modo da rappresentare la velocità “in marcia” del veicolo e non una media diluita dalle soste.

## 5.5 Comunicazione e comandi amministrativi

Il server offre due canali di invio verso i client: invia\_a\_utente (diretto, cerca la sessione per nome nel Registro) e invia\_broadcast (itera su tutte le sessioni attive). Entrambi sono richiamati sia dal terminale comandi della GUI (/all <testo>, /stop, /aiuto) sia dal pannello “Chat privata”, che costruisce internamente lo stesso comando amministrativo Msg riusato dal parser testuale: un solo punto del codice esegue realmente l'invio, evitando due implementazioni parallele che potrebbero divergere.

## 5.6 Interfaccia grafica dell'operatore

La GUI (struct ServerApp) è divisa in quattro pannelli, coerenti con quanto descritto nel manuale utente:

- pannello\_utenti: elenca tutti gli username registrati (letti direttamente dal database) e, per ciascuno, lo stato online/offline e in movimento/fermo, letto dallo snapshot più recente del Registro;
- pannello\_mappa: mostra la mappa centrata sull'utente selezionato, con un plugin grafico che disegna la traccia percorsa e un marcatore colorato (verde in movimento, giallo fermo);
- pannello\_analisi + Chat privata: un unico riquadro di analisi con selettore Giorno/Settimana/Mese, aggiornato automaticamente ogni 2 secondi per l'utente e l'intervallo correnti;
- pannello\_comandi: il terminale testuale per i comandi amministrativi globali.

La GUI non tiene una copia dello stato: ad ogni frame utile interroga il thread di rete tramite ComandoAdminGui::RichiediSnapshot e riceve indietro un evento EventoServerGui::Snapshot con lo stato di tutte le sessioni attive, oltre agli eventi asincroni (righe di log, risultati di analisi, messaggi privati in arrivo) gestiti da processa\_eventi. Questo mantiene la GUI semplicemente “reattiva” a ciò che il thread di rete le comunica, senza replicare la logica di dominio.

## 5.7 Log delle prestazioni (CPU/RAM)

Il task asincrono logger\_cpu, avviato all'avvio del server, usa il crate sysinfo per campionare l'uso di CPU e RAM del processo ogni PERIODO\_LOG\_SEC (120 secondi). Ad ogni campione integra la percentuale di CPU usata nell'intervallo trascorso per ottenere un tempo di CPU cumulativo in secondi, una misura portabile tra Windows, Linux e macOS senza dover leggere direttamente contatori di sistema specifici della piattaforma. Ogni riga scritta in cpu\_log.txt riporta uptime, percentuale di CPU nell'ultimo intervallo, CPU cumulativa, RAM occupata (MB) e numero di client attualmente connessi; ogni riavvio del server aggiunge un'intestazione “=== avvio server ... ===” così il log resta leggibile anche accumulando più esecuzioni nello stesso file.

Un estratto reale del file generato durante un test (due client connessi, secondo campione dopo 4 minuti):

```
[2026-08-05 17:57:30] uptime=2m 00s | CPU intervallo= 0.00% | CPU time totale= 0.000s | RAM= 85.16 MB | client connessi=1
```

```
[2026-08-05 17:59:30] uptime=4m 00s | CPU intervallo= 4.91% | CPU time totale= 5.893s | RAM= 104.05 MB | client connessi=2
```

## 6. Il client

Il binario client emula il terminale di bordo di un singolo veicolo, con la stessa architettura a due thread descritta nel capitolo 2 (GUI su thread principale, rete su thread separato con runtime Tokio a thread singolo, comunicazione tramite due canali).

### 6.1 Architettura interna e concorrenza

Network\_loop apre la connessione TCP verso il server e la divide in due metà (tokio::io::split): un task di lettura dedicato inoltra alla GUI tutti i messaggi che arrivano dal server (inclusi quelli non sollecitati come i messaggi broadcast); il ciclo principale del thread di rete consuma i comandi inviati dalla GUI (ComandoRete) e scrive sul socket attraverso uno scrittore condiviso (Arc<TokioMutex<...>>), necessario perché sia il ciclo comandi sia l'eventuale task di simulazione del percorso devono poter scrivere sullo stesso socket.

Nessuno dei due lati usa unwrap sui canali: se un lato termina (ad esempio la GUI viene chiusa, o la connessione cade), l'altro lo rileva dal fallimento dell'invio/ricezione e reagisce di conseguenza (interrompendo la simulazione in corso, segnalando la disconnessione all'utente), invece di terminare bruscamente il processo.

## 6.2 Autenticazione

La schermata di login/registrazione applica prima i controlli lato client (valida\_username, lunghezza minima della password, corrispondenza tra password e conferma in fase di registrazione) tramite le stesse funzioni del crate shared usate dal server, per dare un riscontro immediato senza un giro di rete; l'hash della password è calcolato subito prima dell'invio con hash\_password, così che il testo in chiaro digitato dall'utente non lasci mai il processo client. L'esito arriva asincronamente come AuthResult/RegisterResult e aggiorna lo stato della GUI (esito\_auth).

## 6.3 Emulazione del movimento

La strategia scelta tra quelle proposte dalla traccia è la lettura di un file di coordinate e tempi (percorso.csv), fornito con un tracciato reale Torino ->Asti: 96 campioni distanziati di 30 secondi, per circa 46,5 km complessivi e 47 minuti e 30 secondi di durata (dati calcolati direttamente dal file fornito). Il file di esempio è anche il dataset di prova/benchmark usato durante lo sviluppo per validare macchina a stati e calcolo dell'analisi con un percorso reale non banale.

Il task simula\_percorso invia un PositionUpdate per il campione corrente, poi attende l'intervallo di tempo dichiarato nel file diviso per il fattore di velocità scelto nel pannello (x1, x10 o x60: x1 rispetta la specifica di un campione ogni 30 secondi reali, i fattori più alti servono solo a velocizzare le dimostrazioni). Il comportamento di "Sosta" è realizzato con un flag atomico condiviso (Arc<AtomicBool>) letto dal task di simulazione: mentre è attivo, l'indice del campione non avanza, quindi il client continua a inviare la stessa posizione, ed è esattamente questo comportamento a far scattare lato server, dopo 180 secondi, la transizione a "fermo". "Ferma" interrompe definitivamente il task (handle.abort()). Se il percorso arriva alla fine, il client resta all'ultimo campione e continua a ritrasmetterlo ogni 30 secondi, come farebbe un veicolo parcheggiato che continua a comunicare la propria posizione.

Il messaggio PercorsoAvviato viene inviato subito prima di avviare il task di simulazione: serve a segnalare al server che la traccia mostrata sulla mappa dell'operatore va ricominciata da questo punto, così un riavvio del percorso non produce sulla mappa una riga di ritorno verso l'inizio del tragitto precedente. Anche lato client la mappa e la telemetria vengono ripulite (traccia.clear(), ultima\_lat/lon a None) nello stesso istante, per coerenza tra le due interfacce.

## 6.4 Interfaccia grafica dell'autista

La GUI (struct `GeoruggineApp`) alterna due viste principali: `vista_login` (schermata di accesso/registrazione) e `vista_principale` (dashboard operativa), mostrata solo dopo un login riuscito. La dashboard è composta da:

- `pannello_telemetria`: stato logico corrente del veicolo (colorato come nel server), ultima posizione inviata, numero di campioni trasmessi; sotto, i controlli di emulazione (selettore di velocità, `Avvia/Sosta/Ferma`) e, in fondo, il riquadro di analisi con selettore `Giorno/Settimana/Mese`;
- `pannello_mappa`: la stessa libreria `walkers` usata dal server, con un plugin analogo (`TracciaVeicolo`) che disegna il tragitto già percorso e la posizione corrente;
- `pannello_messaggi`: cronologia dei messaggi ricevuti dal server (etichettati come `broadcast` o `diretti`) e campo per inviare testo al server.

Per contenere il consumo di CPU (uno dei requisiti espliciti della traccia), la GUI non si ridisegna a frequenza fissa elevata: `request_repaint_after(500ms)` programma un ridisegno al massimo ogni mezzo secondo, mentre gli eventi di rete (arrivo di un messaggio, aggiornamento di stato) richiedono un `repaint` immediato solo quando serve davvero, tramite `ctx.request_repaint()` invocato dalla funzione `notifica`. In pratica la GUI resta ferma quando non c'è nulla di nuovo da mostrare, invece di consumare CPU in un ciclo di disegno continuo a 60 fps senza motivo.

## 6.5 Visualizzazione cartografica

Sia client sia server usano il crate `walkers` per disegnare una mappa `OpenStreetMap` all'interno della finestra `egui`, con un Plugin personalizzato (`TracciaVeicolo` lato client, `TracciaServer` lato server) che, ad ogni frame, proietta le coordinate geografiche in pixel dello schermo (`Projector::project`) e disegna: la spezzata della traccia percorsa, un marcatore verde sul punto di partenza, e un marcatore colorato (verde/giallo) sulla posizione corrente a seconda che il veicolo sia in movimento o fermo. La logica di disegno è pressoché identica nei due binari perché la stessa informazione (traccia + stato) va mostrata in modo coerente sia all'autista sia all'operatore.

## 7. Scelte di progettazione per le prestazioni

La traccia chiede esplicitamente attenzione al consumo di tempo di CPU e alla dimensione dell'eseguibile. Oltre alle scelte già descritte (`repaint` non continuo della GUI, `log` periodico invece che ad ogni campione), il progetto interviene anche a livello di configurazione di build.

### 7.1 Consumo di CPU

- la GUI di entrambi i binari ridisegna solo su richiesta (`request_repaint_after / request_repaint`), non in un ciclo continuo;
- il thread di rete del client usa un runtime `Tokio` a thread singolo, sufficiente per il carico effettivo (una connessione, un task di simulazione).
- il server esegue un solo task per connessione (`I/O-bound`, quindi economico per il runtime asincrono) invece di un thread di sistema operativo per client;
- l'indice `SQLite` su (`id_utente`, `timestamp`) evita scansioni complete della tabella `Posizioni` ad ogni richiesta di analisi o di traccia.

Il file `cpu_log.txt` generato dal server (capitolo 5.7) costituisce la misura richiesta dalla traccia.

## 7.2 Dimensione dell'eseguibile

Il profilo di release del workspace (Cargo.toml) è configurato esplicitamente per ridurre la dimensione del binario finale, oltre a migliorarne la velocità di esecuzione:

```
[profile.release] opt-level = 3 lto = "fat" codegen-units = 1 strip = true incremental = false
```

| Opzione           | Effetto                                                                                                                                                                     |
|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| lto = "fat"       | Link-Time Optimization su tutto il programma: il linker può eliminare il codice morto delle dipendenze (in particolare utile con crate corpose come egui/eframe e walkers). |
| codegen-units = 1 | Compila l'intero crate come una sola unità di generazione codice: più lento da compilare, ma permette ottimizzazioni più aggressive e un binario più compatto.              |
| strip = true      | Rimuove i simboli di debug dal binario finale, che altrimenti gonfierebbero sensibilmente la dimensione dell'eseguibile senza alcun beneficio a runtime.                    |
| opt-level = 3     | Massimizza le ottimizzazioni del compilatore per la velocità di esecuzione (riduce anche il tempo di CPU a runtime).                                                        |

Come richiesto dalla traccia, la dimensione effettiva degli eseguibili prodotti va misurata sulla macchina di build eseguendo (dopo cargo build --release):

```
ls -lh target/release/server target/release/client # Linux/macOS dir
target\release\server.exe target\release\client.exe # Windows
```

server ≈ 13.5 MB

client ≈ 11.7 MB

## 8. Test automatici e dataset di prova

Il progetto include una batteria di test unitari distribuita sui tre crate del workspace (shared, server, client), ciascuno con un proprio modulo #[cfg(test)].

Nel crate shared i test coprono le funzioni di base condivise da client e server: hash verifica che l'hashing della password sia case-insensitive sullo username e sensibile alla password; distanza\_torino\_asti verifica haversine\_km contro la distanza nota tra le due città; username\_validi verifica normalizzazione e vincoli di formato.

Nel crate server, oltre a data\_civile\_nota (conversione giorni-dall'epoca -> data civile, usata da timestamp\_utc per evitare la dipendenza da chrono) e interpreta\_comandi\_base (parsing della barra comandi amministrativa), il modulo di test copre in modo approfondito la funzione analizza(), il cuore del calcolo statistico del server: soglie di movimento/pausa (sopra e sotto i 10 m), lo scarto dei gap di connessione superiori a 5 minuti, e i casi limite di zero o un solo campione (nessuna divisione per zero). Sono inoltre testate le query sul database (elenco utenti, filtraggio temporale della traccia) e, con #[tokio::test], la logica di invio dei messaggi (invia\_a\_utente, invia\_broadcast) e l'esecuzione dei comandi amministrativi (esegui\_comando\_admin).

Nel crate client, i test si concentrano sul parsing del tracciato: `parse_tempo` e `separa_campi` sono verificate su formati validi e non validi, incluso il caso della virgola decimale italiana e dei diversi separatori di campo (;, tab, spazio). È inoltre testata la validazione dei campi di login/registrazione (`valida_campi`), incluso il vincolo sulla lunghezza minima della password e sulla corrispondenza tra password e conferma in fase di registrazione.

Il file `percorso.csv` (Torino -> Asti, 96 campioni, ~46,5 km, ~47 min e mezzo) funge sia da dataset dimostrativo sia da benchmark funzionale end-to-end: eseguendolo a fattore x1 con client e server realmente connessi si possono osservare, su un caso non banale, sia la macchina a stati (transizione a "fermo" imponendo una pausa) sia la coerenza tra il tragitto in km calcolato dal server e la distanza reale nota del percorso Torino-Asti.

Per eseguire l'intera batteria di test automatici:

```
cargo test --workspace
```

## 9. Multiplatforma

La traccia richiede che l'applicazione sia eseguibile su almeno due tra Windows, Linux, macOS, Android, ChromeOS e iOS. Le scelte tecnologiche adottate soddisfano il requisito sulle piattaforme desktop:

- `eframe/egui` è nativamente multiplatforma su Windows, Linux e macOS tramite backend grafici diversi selezionati automaticamente in fase di compilazione;
- Tokio, `rusqlite` e `sha2` non hanno dipendenze specifiche di piattaforma per l'uso fatto in questo progetto;
- `sysinfo` espone un'API uniforme per il monitoraggio di CPU/RAM del processo su Windows, Linux e macOS, evitando codice condizionale per sistema operativo nel logger delle prestazioni;
- le uniche dipendenze dal filesystem (`percorso.csv`, `georuggine.db`, `cpu_log.txt`) usano percorsi relativi portabili, senza separatori o convenzioni specifiche di un solo sistema operativo.

## 10. Limiti noti e possibili sviluppi futuri

- Il canale TCP non è cifrato: username, password (già sottoposta a hash, ma comunque non su canale protetto) e posizioni viaggiano in chiaro. Un'estensione naturale sarebbe l'introduzione di TLS (ad es. tramite `rustls`).
- L'hashing della password con SHA-256 salato allo username è adeguato al contesto del progetto ma non equivalente, in termini di resistenza ad attacchi a forza bruta, ad algoritmi pensati apposta per le password (`bcrypt`, `scrypt`, `argon2`).
- È supportata una sola sessione attiva per account: un secondo login con lo stesso username viene rifiutato finché il primo resta connesso, coerentemente con l'idea di un solo veicolo per autista.
- La strategia di emulazione del movimento implementata è la lettura di un file di coordinate e tempi;
- La cronologia della chat privata lato server è mantenuta solo in memoria (nella GUI) e non viene persistita nel database: viene perduta al riavvio del server.

- Il file `cpu_log.txt` cresce in modo indefinito nel tempo (append-only, senza rotazione): per esecuzioni molto lunghe andrebbe introdotta una politica di rotazione o troncamento.

## 11. Conclusioni

Georuggine implementa tutti i requisiti funzionali richiesti dalla traccia con un'architettura simmetrica tra client e server che ha permesso di riusare la stessa logica di validazione, hashing e calcolo geografico in entrambi i binari tramite il crate shared. Le scelte di configurazione del profilo di release e di gestione del repaint della GUI rispondono esplicitamente al requisito di attenzione alle prestazioni, misurabile concretamente tramite il log periodico generato dal server.